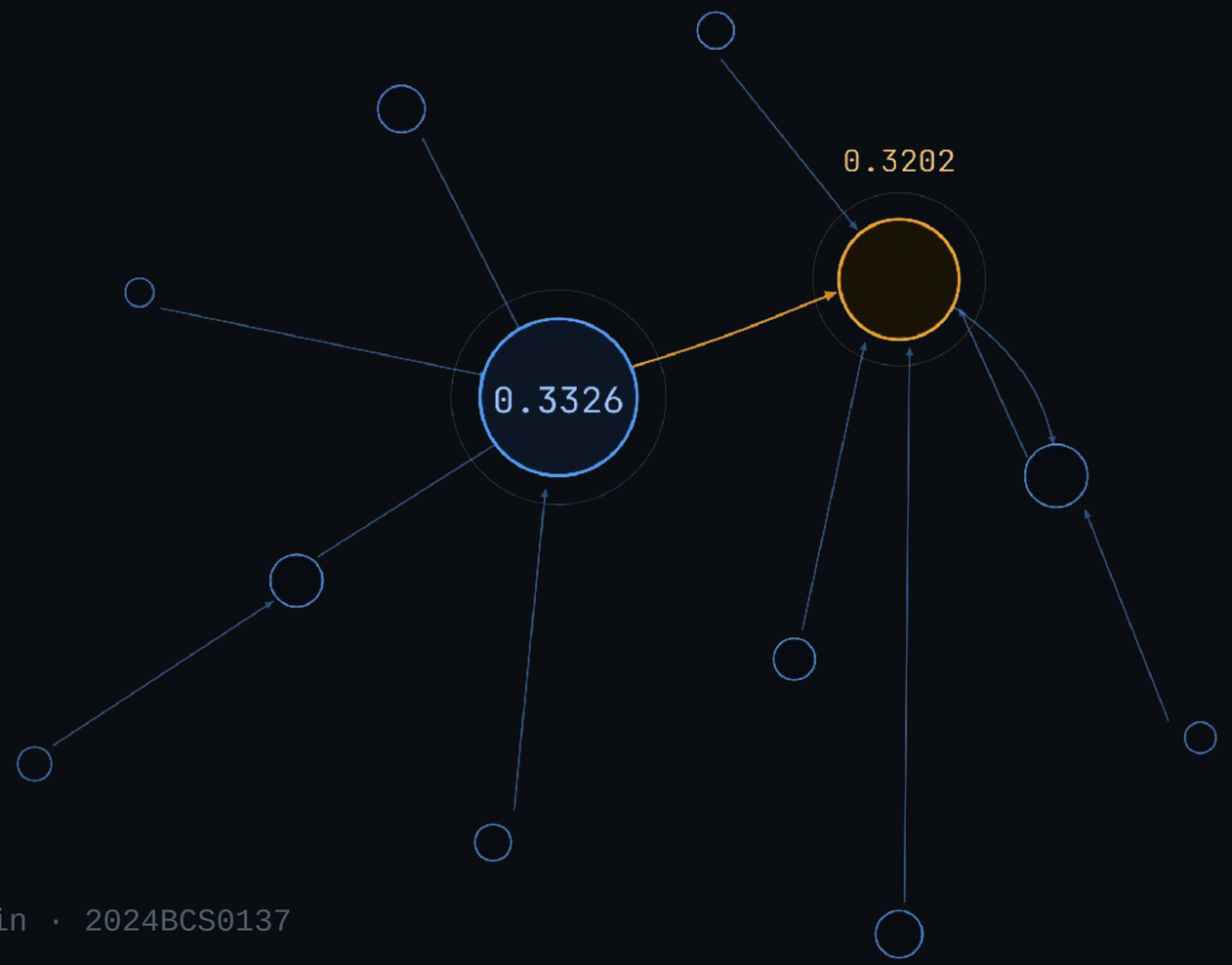
CSS 311 · PARALLEL AND DISTRIBUTED COMPUTING

PageRank Algorithm

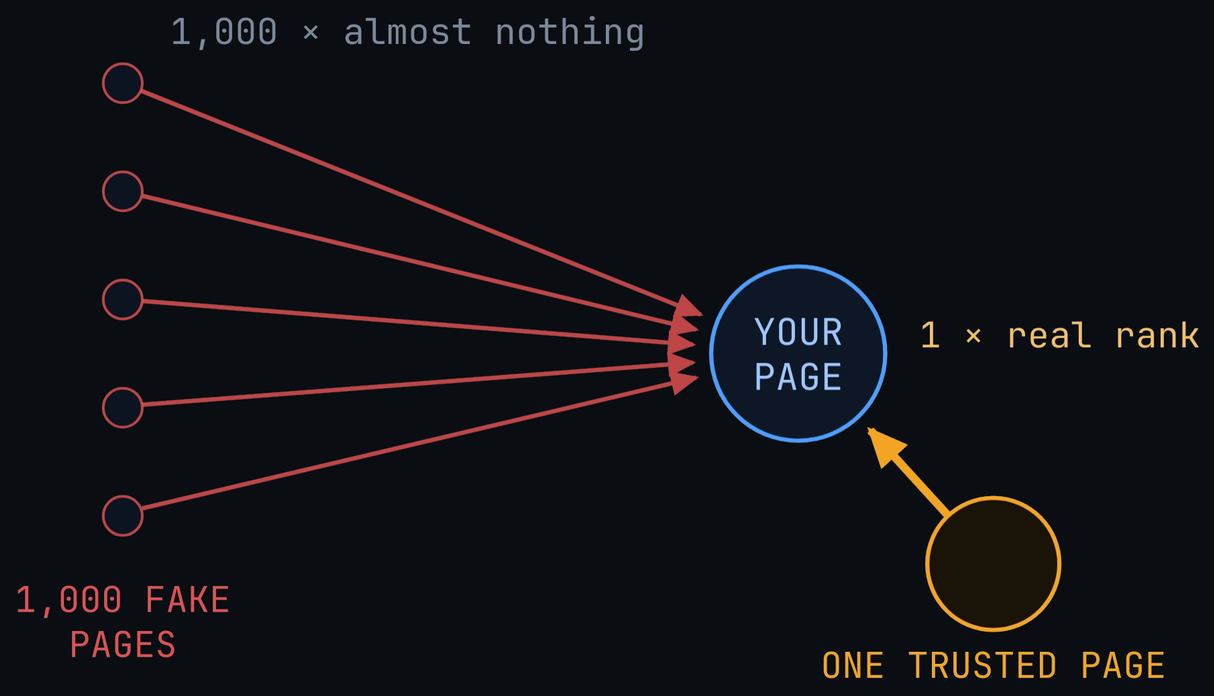
From Serial Recurrence to OpenMP Parallel Design



Assignment I · Module I & II

Niranjan R Soorej · 2024BCS0117 · Mohammed Ashar Husain · 2024BCS0137

A link is a vote — but not every vote is worth the same



01 A link is a vote for another page.

02 A vote from an important page counts for more.

03 A page splits its importance across its out-links — so voting costs rank.

THE CHEAT

Build a thousand pages of your own and point them all at yourself.

THE FIX

Nobody links to those pages, so they hold almost no rank to give away. A thousand empty votes stay worth almost nothing.

And 15% of the time the surfer ignores links entirely and jumps to a random page — so a closed loop of pages cannot hoard rank either. That 15% is the damping factor `d = 0.85` used for the rest of this deck.

ROADMAP

One path, six stages



Web Graph
Problem & objective

Rank Recurrence
Serial algorithm & complexity

PCAM Design
Foster's methodology

Iterations
Two graphs, hand-traced

OpenMP
Regions, reduction & sync

Performance
Timing, speedup & efficiency

So… what makes a web page important?



HOW TO READ THIS PICTURE

01 Every circle is a web page. Every arrow is a link from one page to another.

02 Importance flows along the arrows , in the direction they point.

03 A page with two links out gives each one half of its own rank — a vote is not free.

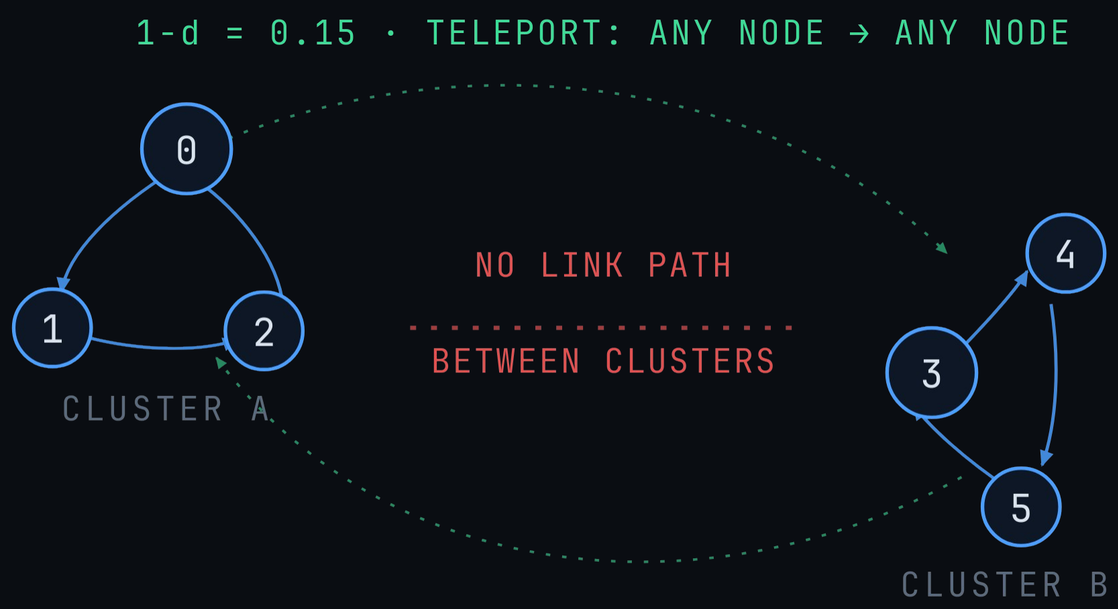
04 Page A is large because five pages point at it. Page B has one link, from a page that is itself small.

WHAT	Compute the rank of every node in a directed graph.
WHY	Real graphs are huge and the ranks are recomputed many times — so parallelise it with OpenMP.
WHERE	Search ranking, citation analysis, any link structure.

PageRank turns link structure into a numerical measure of importance.

WHY THE DAMPING FACTOR EXISTS

Without teleportation, disconnected pages are unreachable



01 A pure link-following walk (d=1) can get permanently trapped inside whichever component it starts in.

02 A disconnected graph's transition matrix is reducible — it has no single, well-defined stationary distribution to converge to.

03 The fix: with probability 1−d, ignore links entirely and jump to a uniformly random node anywhere in the graph.

04 That always-on term, `(1−d)/N` , is added identically to every node — it stitches disconnected clusters into one chain and guarantees convergence.

Damping isn't only for dangling nodes — it's what makes PageRank well-defined on any graph, connected or not.

The core idea: rank flows through links



0.85 damping factor d

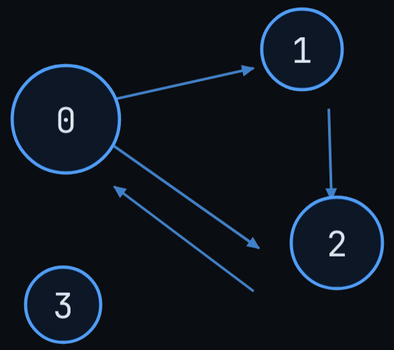
```
Initialize PR[i] = 1/N
repeat
  dangling = sum PR[j], for outdeg(j) = 0
  for each node i:
    newPR[i] = (1-d)/N + d*dangling/N
    for each j -> i:
      newPR[i] += d*PR[j]/outdeg(j)
  error = sum |newPR[i] - PR[i]|
  PR = newPR
until error < eps
```

KEY PARAMETERS

d	Damping factor (0.85)
N	Number of nodes
outdeg(j)	Out-degree of node j
ε (eps)	Convergence threshold

Dangling nodes (outdeg = 0) redistribute their rank equally across all N nodes — otherwise rank "leaks" out of the system.

Now split the work



ONE GRAPH · N NODE UPDATES

→

THREAD 0 nodes { 0, 1 }

THREAD 1 nodes { 2, 3 }

static schedule · shared PR[] and graph · reductions for dangling & error

P Partitioning

- Data decomposition
- Divide nodes among threads
- Same computation on each chunk

C Communication

- Shared PR[] and graph
- Global: dangling sum & error
- Use reduction for global values

A Agglomeration

- Group nodes into chunks
- Reduce parallelization overhead

M Mapping

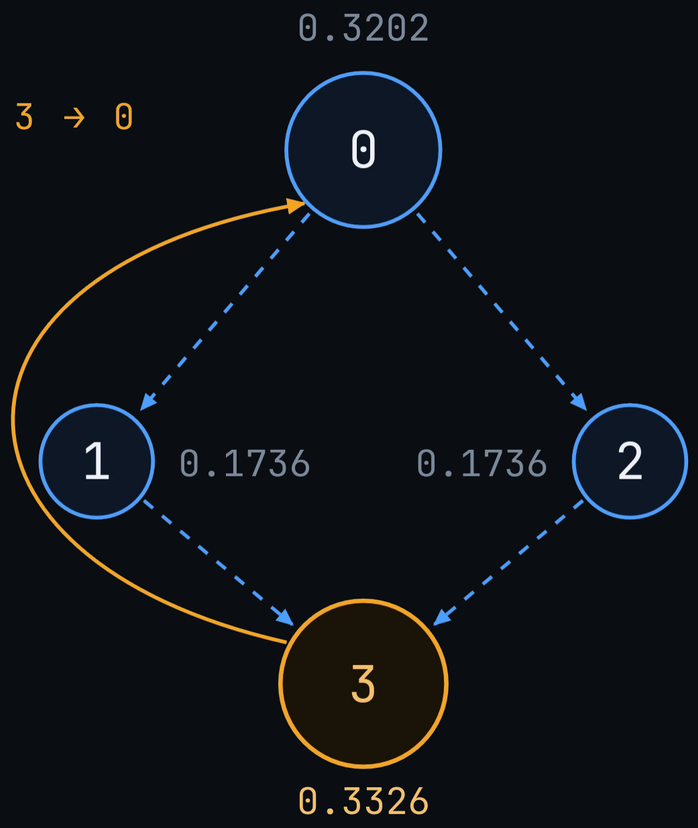
- Map chunks to OpenMP threads
- Static scheduling
- Fixed node ranges per thread

SOLUTION DEMONSTRATION

MODULE I · Q3 — WORKED EXAMPLE 1

N = 4 · d = 0.85 · no dangling node

Example 1 — watch the rank move



outdeg: 0:2 · 1:1 · 2:1 · 3:1 — node area ∝ converged rank

INITIAL STATE PR = (0.25, 0.25, 0.25, 0.25)

ITERATION 01

NODE	UPDATE	VALUE
PR[0]	0.0375 + 0.85×(PR[3]/1)	0.2500
PR[1]	0.0375 + 0.85×(PR[0]/2)	0.1438
PR[2]	0.0375 + 0.85×(PR[0]/2)	0.1438
PR[3]	0.0375 + 0.85×(PR[1]/1 + PR[2]/1)	**0.4625**

error after iteration 1 = 0.4250

ITERATION 02

PR = (0.4306, 0.1438, 0.1438, 0.2819) · error = 0.3613

CONVERGED **81** iterations

PR[0]	PR[1]	PR[2]	PR[3]
0.3202	0.1736	0.1736	0.3326

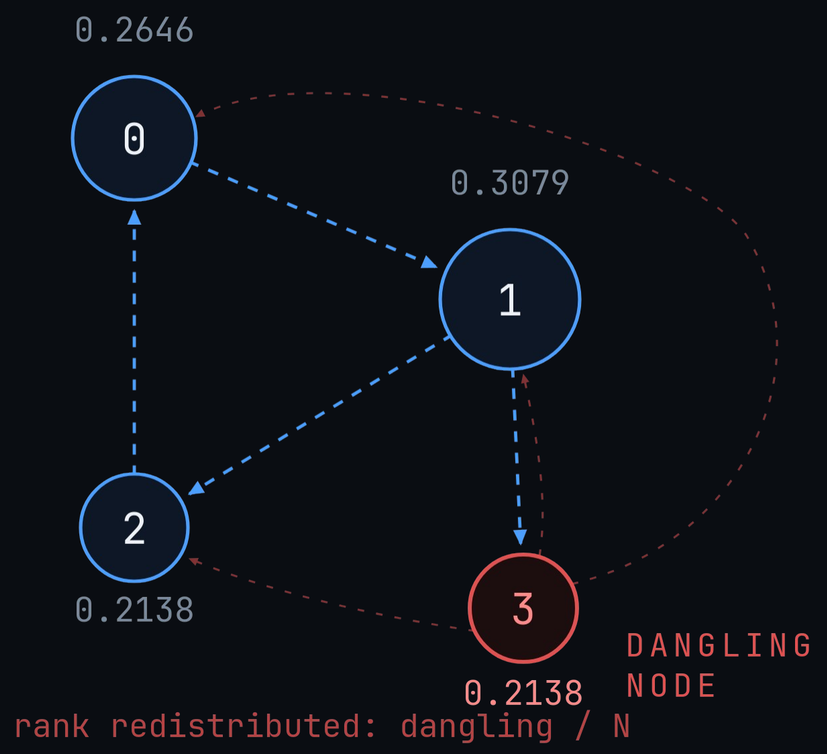
Node 3 receives the full rank of both 1 and 2, then hands it back to the hub — so the loop settles with 0 and 3 sharing most of the mass.

SOLUTION DEMONSTRATION

MODULE I · Q3 — WORKED EXAMPLE 2

N = 4 · d = 0.85 · outdeg(3) = 0

Example 2 — what happens when a page has nowhere to go?



outdeg: 0:1 · 1:2 · 2:1 · 3:0

ITERATION 01 base = 0.0375 · dangling term = 0.053125

NODE	UPDATE	VALUE
PR[0]	base + 0.85×PR[2] + dngl	0.3031
PR[1]	base + 0.85×PR[0] + dngl	0.3031
PR[2]	base + 0.85×(PR[1]/2) + dngl	0.1969
PR[3]	base + 0.85×(PR[1]/2) + dngl	0.1969

error after iteration 1 = 0.2125

ITERATION 02

PR = (0.2467, 0.3370, 0.2082, 0.2082) · error = 0.1129

CONVERGED **32** iterations

PR[0]	PR[1]	PR[2]	PR[3]
0.2646	0.3079	0.2138	0.2138

Without redistribution
rank leaks

With redistribution
rank conserved

MODULE I · Q4–Q5 — SERIAL IMPLEMENTATION

Serial PageRank: input size vs. runtime

Input: node count N and generated edge count E. Hold graph-generation rule and ε constant.

PER ITERATION
O(N + E)

TOTAL
O(I · (N + E))

SPACE
O(N + E)

Input density: about 6 edges/node (existing pattern)

OBSERVATION TABLE · N is the experiment input

Input nodes N	Generated edges E	Iterations	Serial time (s)	Result status
1,000	5,994	13	0.000120	Measured
10,000	59,995	13	0.001672	Measured
100,000	599,992	13	0.020366	Measured
1,000,000	**≈ 6,000,000**	**Measure**	**Measure**	**Run this input**
10,000,000	**≈ 60,000,000**	**Measure**	**Measure**	**Large input; check RAM**

How to run: change N as an input parameter; generate/build the graph, then time the serial algorithm.

Record compiler flags, ε, iteration count, CPU, and median of repeated runs. Do not treat proposed rows as measured.

The good news: every node can be updated independently



> Node updates within one iteration are parallel, but iterations themselves remain sequential.

SHARED

PR[], graph / in-link adjacency (read-only during the loop).

PRIVATE

Loop variable i, local sum accumulator.

REDUCTION

Dangling sum, error norm.

SYNCHRONIZATION

Barrier required before the next iteration begins.

Turning the recurrence into OpenMP

```
Initialize PR[i] = 1/N
repeat
  #pragma omp parallel for reduction(+:dangling)
    dangling = sum PR[j], outdeg(j)=0
  #pragma omp parallel for
    for each i: compute newPR[i]
  barrier
  #pragma omp parallel for reduction(+:error)
    error = sum |newPR[i]-PR[i]|
  PR = newPR
until error < eps
```

01 Calculate dangling sum
02 Parallelize node updates
03 Synchronize
04 Calculate error
05 Swap PR / newPR
06 Repeat until converged

STATIC PARTITION · 2 THREADS · NODES 0–3

T0	T1
{ 0, 1 }	{ 2, 3 }

BARRIER

Parallel result = serial result

PR = (0.2646, 0.3079, 0.2138, 0.2138)

32 iterations — identical to serial

PageRank OpenMP: Synchronization Strategy Benchmark

N = 10,000 • 4 threads • 55,000 edges • tolerance = 10^{-8} • 8 iterations

No.	Dangling Sum	Error Sum	Runtime (s)	Iterations
1	Reduction	Reduction	0.00165437	8
2	Critical	Critical	0.02187610	8
3	Atomic	Atomic	0.01539558	8
4	Reduction	Critical	0.01919751	8
5	Reduction	Atomic	0.01346428	8
6	Critical	Reduction	0.00179802	8
7	Critical	Atomic	0.01527215	8
8	Atomic	Reduction	0.00231659	8
9	Atomic	Critical	0.01645321	8

WHAT THE RESULTS SAY

FASTEST OBSERVED

Reduction + Reduction · 0.00165437 s

SLOWEST OBSERVED

Critical + Critical · 0.02187610 s

INTERPRETATION

Reduction + Reduction is the recommended baseline: it avoids per-update locking. The fastest single run is not proof of a generally faster method.

Important: this graph has no dangling nodes, so the dangling-sum loop performs no shared updates. Repeat runs on a graph with dangling nodes for a fairer comparison.

PRACTICAL TAKEAWAY

REDUCTION + REDUCTION

Best default to benchmark

Low synchronization overhead

ATOMIC + REDUCTION

Fastest in this run

Validate with repeated runs

CRITICAL + CRITICAL

Slowest in this run

Frequent serialized updates

Use identical inputs and compiler flags; report median runtime across 7–10 runs. Do not alter measured values to fit an expected result.

PageRank: Serial vs Parallel Performance

Measured results across six graph sizes

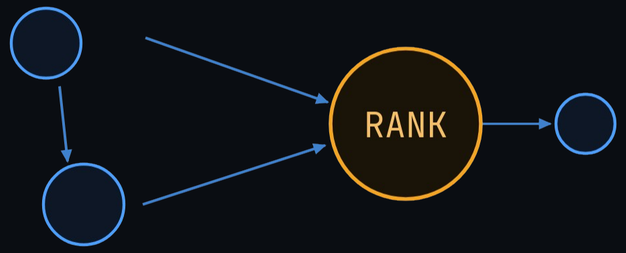
Input nodes (N)	Threads (P)	Serial time (s)	Parallel time (s)	Speedup	Efficiency
5	2	0.00000019	0.00025754	**0.0007×**	0.04%
1,000	2	0.00010956	0.00020852	**0.53×**	26.27%
10,000	4	0.00054458	0.00061081	**0.89×**	22.28%
100,000	8	0.00483282	0.00204145	**2.37×**	29.60%
1,000,000	16	0.04661068	0.01645861	**2.83×**	17.70%
10,000,000	16	0.49055496	0.18424764	**2.66×**	16.64%

SERIAL `./pagerank_serial N` • **PARALLEL** `./pagerank_openmp N P`

Speedup = Serial time ÷ Parallel time | Efficiency = Speedup ÷ Threads × 100%

CONCLUSION

PageRank: from links → rank → parallel work



GRAPH → RECURRENCE → ITERATIONS → CONVERGENCE → OPENMP → PERFORMANCE

Correctness first

Serial and OpenMP results match exactly on both test graphs.

PCAM gave structure

Partitioning + shared data + reductions + static mapping.

Scaling is data-dependent

Runtime follows O(I·(N+E)); observed speedup is capped by available cores, not the algorithm.

Dangling nodes matter

Redistribution keeps total rank conserved across iterations.

PageRank + Graph ML in the modern world

PageRank measures network importance; Graph ML uses that signal to make better predictions.

01 HOW THEY WORK TOGETHER

PageRank scores identify influential nodes. Graph ML combines these scores with node features and connections to learn patterns.

02 SOCIAL NETWORKS

Find influential accounts, suggest relevant people or communities, and improve content recommendations using relationship patterns.

03 WEB SEARCH & DISCOVERY

Link importance can help rank useful pages and entities, alongside content quality, freshness, and many other signals.

04 FRAUD & SPAM DETECTION

Graph patterns can expose suspicious account clusters, coordinated activity, and links to known risky entities.

05 RECOMMENDATIONS

Use graph relationships and importance scores to support product, video, paper, or friend recommendations.

06 WHY IT MATTERS TODAY

Digital services are interconnected. Graph ML turns those connections into signals for ranking, discovery, risk analysis, and prediction.

KEY IDEA: PageRank provides an importance signal; Graph ML learns from that signal plus the wider graph.

PageRank is one useful feature—not a complete ranking or decision system on its own.

Thank you.

